

pol_strips_plot

September 10, 2025

```
[1]: # Analysing data for CygX1
```

```
[2]: %matplotlib inline
import numpy as np
from uncertainties import ufloat
from uncertainties import umath
import matplotlib.pyplot as plt
from scipy import optimize, interpolate
from scipy.special import erf
from scipy.optimize import minimize
import scipy.integrate as integrate
import aplpy
from astropy import stats
import pickle
```

```
[3]: class star:
    def __init__(self, PD, PDerr, PA, PAerr):
        self.PD = PD          # in %
        self.PDerr = PDerr
        self.PA = PA          # in deg
        self.PAerr = PAerr
```

```
[4]: class Obs():
    def __init__(self, name, JD, q, qerr, u, uerr, x=0.0, y=0.0):
        self.name = name
        self.JD = JD
        self.q = ufloat(q, qerr)
        self.u = ufloat(u, uerr)
        self.x = x
        self.y = y

    def getP(self):
        return umath.sqrt(self.q ** 2 + self.u ** 2)

    def getPA(self):
        PAval = 0.5*umath.atan2(self.u, self.q)
        PAunc = self.getSigma(self.getP().n, self.getP().s)
        return ufloat(np.degrees(PAval.n), np.degrees(PAunc))
```

```

def correctInst(self,q_inst,u_inst):
    self.q = self.q - q_inst
    self.u = self.u - u_inst

def correctRot(self,dPA):
    THETA = umath.radians(dPA)
    Qnew = self.q * umath.cos(2*THETA) - self.u * umath.sin(2*THETA)
    Unew = self.q * umath.sin(2*THETA) + self.u * umath.cos(2*THETA)
    self.q = Qnew
    self.u = Unew

def EVPA_pdf(self,theta,P0):
    """
    EVPA measurements are also non-Gaussian and defined by the following
    probability density (Naghizadeh-Khouei & Clarke 1993):
    """
    g = 1/np.sqrt(np.pi)
    ita0 = float(P0)/np.sqrt(2) * np.cos(2 * theta)
    g = g * (g + ita0 * np.exp(ita0**2) * (1 + erf(ita0)))
    g = g * np.exp(-(float(P0)**2)/2)
    return g

def int_eq(self,sigma,snr):
    """ This is the integral of EVPA probability density from -sigma to_
    ↪sigma """
    integ = integrate.quad(lambda x: self.EVPA_pdf(x,snr),-sigma,sigma)
    return abs(integ[0] - 0.68268949)

def getSigma(self,pd,pd_err):
    snr = pd/pd_err
    if snr > 20:
        # it is a good approximation even for snr = 5
        return 0.5*1.0/float(snr)

    if snr < np.sqrt(2.0):
        pd = 0.0
    else:
        pd = np.sqrt(pd**2 - pd_err**2)

    snr = pd/pd_err

    res = minimize(self.int_eq, [np.pi/50], args=(snr,),_
    ↪method='Nelder-Mead', tol=1e-5) # np.pi/50 = 3.6 deg - just a reasonable_
    ↪guess
    if res.status != 0:
        print('Something is wrong with the EVPA uncertainty calculation:\n')

```

```

        return np.nan

    return res.x[0]

class Stand():
    def __init__(self, st):
        self.name = st
        stand = stand_pol.STANDARDS[self.name]
        self.PD = ufloat(stand.PD/100., stand.PDerr/100.)
        self.PA = ufloat(stand.PA, stand.PAerr)
        self.calcQU()

    def calcQU(self):
        self.q = self.PD * umath.cos( 2 * umath.radians(self.PA) )
        self.u = self.PD * umath.sin( 2 * umath.radians(self.PA) )

```

Raw processed data (no model):

```

[5]: gfs1 = Obs("CygX1_gfs1", 2459713.4834928, 0.00945, 0.00108, -0.05371, 0.00105,
    ↪1032.2, 1030.6)
gfs2 = Obs("CygX1_gfs2", 2459714.5546300, 0.01089, 0.00082, -0.03957, 0.00042,
    ↪1028.4, 1031.0)
gfs3 = Obs("CygX1_gfs3", 2459730.5692164, 0.02152, 0.00120, -0.04535, 0.00116,
    ↪1032.9, 1033.1)
gfs4 = Obs("CygX1_gfs4", 2459731.5190509, 0.01049, 0.00451, -0.00066, 0.00430,
    ↪1026.9, 1027.1)
gfs5 = Obs("CygX1_gfs5", 2459732.4071788, 0.00258, 0.00269, -0.00844, 0.00267,
    ↪1029.4, 1028.6)
gfs6 = Obs("CygX1_gfs6", 2459732.4558553, 0.00549, 0.00106, -0.05031, 0.00105,
    ↪1021.7, 1032.4)
gfs7 = Obs("CygX1_gfs7", 2459732.4731289, -0.02550, 0.00253, -0.04658, 0.00246,
    ↪1030.4, 1031.9)
gfs8 = Obs("CygX1_gfs8", 2459736.4691250, -0.00053, 0.00177, -0.05485, 0.00173,
    ↪1029.9, 1028.6)

```

```

[6]: def coordinates_os_stars():
    ra, dec, Ps, angles = [], [], [], []

    x,y = 299.608150,35.199483
    PD = gfs1.getP()
    EVPA = gfs1.getPA()
    ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

    x,y = 299.582586,35.185228
    PD = gfs2.getP()
    EVPA = gfs2.getPA()
    ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

```

```

x,y = 299.632867,35.213000
PD = gfs3.getP()
EVPA = gfs3.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

x,y = 299.594674,35.248638
PD = gfs4.getP()
EVPA = gfs4.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

x,y = 299.638525,35.227587
PD = gfs5.getP()
EVPA = gfs5.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

x,y = 299.513901,35.192384
PD = gfs6.getP()
EVPA = gfs6.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

x,y = 299.658824,35.142903
PD = gfs7.getP()
EVPA = gfs7.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

x,y = 299.561068,35.122415
PD = gfs8.getP()
EVPA = gfs8.getPA()
ra.append(x), dec.append(y), angles.append(EVPA.n), Ps.append(PD.n)

""" Convert angles from range [-pi,pi] or [-pi/2,pi/2]
    to [0,pi).
    """
for aa in range(len(angles)):
    if angles[aa] < 0:
        angles[aa] = 180 - abs(angles[aa])
    if angles[aa] == 180:
        angles[aa] = 0
    #angles[aa] = angles[aa]*180.0/np.pi

return ra, dec, Ps, angles

```

```

def segments_on_map(fitsfile, ra, dec, Ps, pas, scale = 1000):
    '''
    Plot polarization segments on fits file.
    Input:
    fitsfile: string, name of fits file
    ra: list of ra
    dec: list of dec
    pas: list of angles in radians. Angles are with respect to y axis of
    ↪ image.
        You need to rotate the real angle to have angles with respect to
    ↪ north.
    savename: string, name of plot to be saved
    coord: coordinate system of fits image
    scale: float/int, determines how long the segments will be in pixels
    '''
    fig = aplpy.FITSFigure(fitsfile, figsize = (9,9))
    fig.show_grayscale(invert=True)
    fig.add_grid()
    #fig.grid._grid._linewidths = (0.4,)
    fig.grid.show()

    linelist1 = []
    kukloi_ra, kukloi_dec, kukloi_polosi = [], [], []
    for iv in range(len(pas)):
        xpix,ypix=fig.world2pixel(ra[iv],dec[iv])
        linelength_half= scale*Ps[iv]

        y=[ypix-linelength_half*np.cos(pas[iv]),
           ypix+linelength_half*np.cos(pas[iv])]
        x=[xpix+linelength_half*np.sin(pas[iv]),
           xpix-linelength_half*np.sin(pas[iv])]
        x_world,y_world=fig.pixel2world([x[0],x[1]], [y[0],y[1]])
        line=np.array([x_world,y_world])
        linelist1.append(line)

    fig.show_lines(linelist1[:-1], layer='line', color='r', linewidths=1.3)
    fig.show_lines([linelist1[-1]], layer='line1', color='cyan',
    ↪ linewidths=1.3)
    #fig.show_ellipses([35.3125], [25.3], 0.09, 0.07, angle=17,
    ↪ layer='ellipse', color='g')
    fig.add_label(299.54,35.23,'5%', size=22, color='r')
    fig.add_label(299.608150+0.01,35.199483,'gfs1', size=20, color='r')

```

```

fig.add_label(299.582586-0.01,35.185228,'gfs2', size=20, color='r')
fig.add_label(299.632867+0.01,35.213000,'gfs3', size=20, color='r')
fig.add_label(299.594674+0.01,35.248638,'gfs4', size=20, color='r')
fig.add_label(299.638525-0.01,35.227587,'gfs5', size=20, color='r')
fig.add_label(299.513901+0.01,35.192384,'gfs6', size=20, color='r')
fig.add_label(299.658824-0.005,35.139,'gfs7', size=20, color='r')
fig.add_label(299.561068-0.01,35.122415,'gfs8', size=20, color='r')
fig.add_label(299.5903125-0.017,35.20160667,'CygX1', size=20, color='r')
return

eikona_fits = 'dss2.19.58.21.67+35.10.55.78.fits'
ras_all, decs_all, Ps, angles = coordinates_os_stars()
angles = np.array(angles)
angles = np.radians(angles)
ra = ras_all
dec = decs_all
# Plot polarization segments on DSS
coordang = 0.
pas_forplot = angles + coordang

segments_on_map( eikona_fits, ra, dec, Ps, pas_forplot, scale = 789)

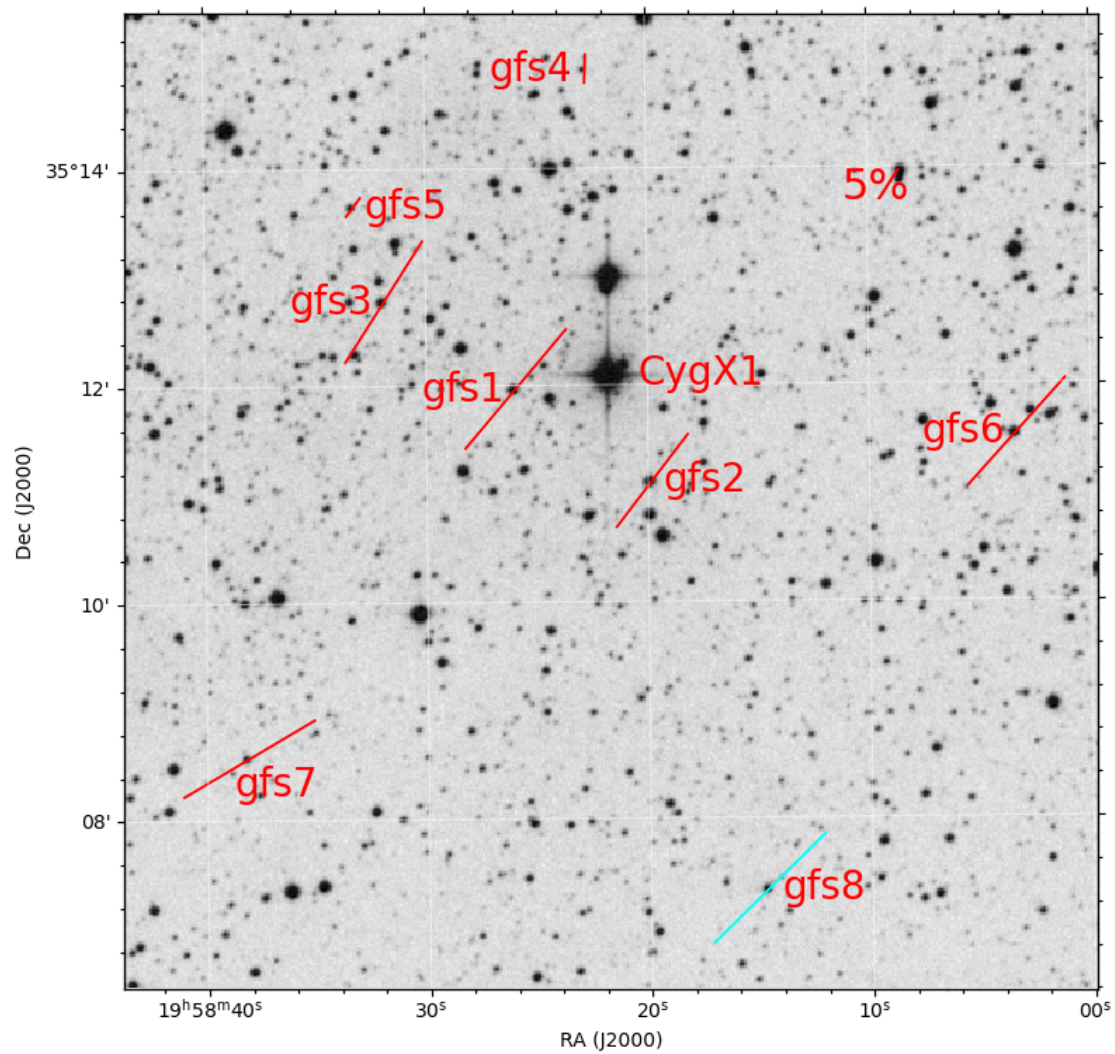
```

```

WARNING: VerifyWarning: Verification reported errors: [astropy.io.fits.verify]
WARNING: VerifyWarning: Card 'SKEW' is not FITS standard (invalid value string:
'5.5002485391277E-01, 5.0445841343823E-01 /Measure of skew'). Fixed 'SKEW'
card to meet the FITS standard. [astropy.io.fits.verify]
WARNING: VerifyWarning: Note: astropy.io.fits uses zero-based indexing.
[astropy.io.fits.verify]
WARNING: FITSFixedWarning: PC001001= 9.9996124099020E-01 /PC matrix
this form of the PCi_ja keyword is deprecated, use PCi_ja. [astropy.wcs.wcs]
WARNING: FITSFixedWarning: PC001002= 9.5982825921584E-03 /PC matrix
this form of the PCi_ja keyword is deprecated, use PCi_ja. [astropy.wcs.wcs]
WARNING: FITSFixedWarning: PC002001= -8.8055523939399E-03 /PC matrix
this form of the PCi_ja keyword is deprecated, use PCi_ja. [astropy.wcs.wcs]
WARNING: FITSFixedWarning: PC002002= 9.9995392280500E-01 /PC matrix
this form of the PCi_ja keyword is deprecated, use PCi_ja. [astropy.wcs.wcs]
WARNING: FITSFixedWarning: 'datfix' made the change 'Set MJD-OBS to 48473.000000
from DATE-OBS.
Changed DATE-OBS from '1991/08/05' to '1991-08-05''. [astropy.wcs.wcs]
WARNING: AstropyDeprecationWarning: CoordinateHelper.ticks should not be
accessed directly and is deprecated
[astropy.visualization.wcsaxes.coordinate_helpers]

INFO: Auto-setting vmin to 1.133e+04 [aplpy.core]
INFO: Auto-setting vmax to 2.829e+04 [aplpy.core]

```



[]: